

Table of Contents

1. Executive Summary and Gate Verdict	1
2. Verification Scope and Methodology	2
3. Technology Stack Verification	3
4. Architecture and Access Control Verification	4
5. Security and Data Protection Verification	6
6. Database, Money and Ghana Data Verification	7
7. API, Storage and Environment Verification	9
8. Frontend and Test Suite Results	11
9. Issues Register	12
10. Production Risks	13
11. Phase 2 Readiness	14

1. Executive Summary and Gate Verdict

109 / 109 Independent adversarial checks passed	48 / 48 Foundation verification suite passed	64 / 64 RBAC escalation pairs correct	10 Issues registered (0 critical)
--	---	--	---

This report documents an independent engineering gate review of the Dwellers Phase 1 foundation, performed against the actual codebase rather than against the Phase 1 self-report. The review covered fourteen verification areas: declared stack, layered architecture, role-based access control, security posture, database schema and PostgreSQL readiness, monetary handling, Ghana market data, API conventions, file storage, environment configuration, frontend behaviour, test suites, and repository hygiene. Every claim made by the Phase 1 report was re-derived from source files, live HTTP traffic, browser sessions, and a purpose-built adversarial test script.

The headline conclusion is positive: the Phase 1 implementation genuinely exists and behaves as reported. The authorization engine enforces all sixty-four actor-to-target escalation rules correctly, guards fail closed with typed 401 and 403 errors, the storage driver rejected every path-traversal probe at runtime, monetary arithmetic is integer-exact, no secrets were found in the working tree or in repository history, and the frontend renders cleanly on desktop and mobile with zero console errors. The project's own verification suite reproduces 48 of 48 passes, and lint is clean.

The gate verdict is therefore **PASS WITH CONDITIONS** rather than a clean pass. Two conditions dominate. First, the TypeScript build deliberately ignores type errors, and the underlying count is worse than the documented debt suggests: tsc currently reports roughly sixteen errors, including missing type imports inside the core authorization and audit modules. These errors have no runtime effect because types are erased at execution, but they mean type safety is not actually enforced at build time. Second, a real .env file has been tracked in git since the initial scaffold commit; it contains only a local SQLite path, so it is not a credential leak, but it defeats the repository's own ignore policy. Both conditions are cheap to fix and are specified, with five further minor conditions, in Chapter 9. None of them requires redesign, and all of them can be absorbed into the start of Phase 2.

Dimension	Result	Evidence
Independent adversarial suite	109 / 109 passed	scripts/gate-review.ts (15 sections)
Project verification suite	48 / 48 passed	bun run verify
ESLint	Clean	bun run lint, zero findings
TypeScript tsc --noEmit	FAIL - approx. 16 errors	Masked by ignoreBuildErrors flag
Live API probes	Healthy	/api/health 200, database connected
Browser E2E	Zero errors	Desktop 1440px and mobile 390px
Git state	Clean tree, 4 conventional commits	No commits created during review
Gate verdict	PASS WITH CONDITIONS	Conditions listed in Chapter 9

Table 1.1 - Gate summary

2. Verification Scope and Methodology

The review was structured around the fourteen areas requested for the gate, and it deliberately avoided trusting any prior report. Instead of re-reading the Phase 1 summary and confirming impressions, the reviewer re-derived every fact from four kinds of evidence: direct inspection of source files, execution of a new adversarial test script written for this review, live HTTP and browser sessions against the running application, and repository forensics including full git history.

The centrepiece of the method is **scripts/gate-review.ts**, a 109-assertion adversarial suite written independently of the project's own verification script. Where the project suite confirms design intent, the gate suite probes for weaknesses: it walks the complete eight-by-eight privilege-escalation matrix rather than sampling it, forces guard functions down their failure paths, attempts filesystem traversal against a temporary storage sandbox, feeds malformed hashes and oversized bodies into the validation layer, intercepts logger output to confirm secret redaction, and greps the source for float-based money arithmetic. Four initial failures were all traced to test harness artifacts or documented development-mode behaviour, and were re-verified accordingly; the suite finished at 109 of 109 without any project code being modified.

- Stack: versions read from installed packages (bun pm ls), tsconfig and toolchain output.
- Architecture: module layout, handler pipeline order proven by source indexing; envelope discipline proven by repository-wide search.
- RBAC: full 8 x 8 escalation matrix, guard throw-paths, permission matrix boundaries, wildcard semantics.

- Security: password hashing, rate limiting (subprocess with limiter enabled), headers on live responses, upload policy and runtime traversal, log redaction, error mapping, secret sweeps of tree and history.
- Data: schema inspection, PostgreSQL blocker sweep, integer money precision probes, Ghana region and phone validation probes including adversarial inputs.
- Delivery: live curl probes, browser desktop and mobile passes with console capture, lint, typecheck, both test suites, git status and history review.

Artifacts produced by the review are limited to the audit script and screenshots under `download/gate-review/`. No project source file was changed, no commit was created, and no feature work was performed, in line with the review's terms of reference.

3. Technology Stack Verification

Declared versions were checked against the lockfile-resolved packages actually installed, not against `package.json` ranges. Every element of the reported stack is present and current. The runtime is Bun 1.3.14 with the Next.js 16.1.3 App Router convention, including the `proxy.ts` network layer that replaced the deprecated middleware convention during Phase 1.

Layer	Declared	Installed	Verification
Framework	next ^16.1.1	16.1.3	App Router, <code>proxy.ts</code> convention active
Language	typescript ^5	5.9.3	strict: true in <code>tsconfig</code>
Styling	tailwindcss ^4	4.1.18	PostCSS plugin, tokens in <code>globals.css</code>
UI kit	shadcn/ui	new-york	<code>components.json</code> plus 47 ui components
ORM	prisma ^6.11.1	6.19.2	SQLite datasource, schema pushed
Auth	next-auth ^4.24.11	4.24.13	JWT strategy wired via session abstraction
Validation	zod ^4.0.2	4.3.5	env, API bodies, queries, upload policies
Lint	eslint ^9	9.39.2	Flat config; clean run
Runtime	Bun	1.3.14	Scripts, tests and server all run under Bun

Table 3.1 - Verified stack (installed versions)

Two configuration caveats qualify the otherwise strict TypeScript posture. The Next config sets `typescript.ignoreBuildErrors` to true, documented as inherited technical debt; this review found the real error count behind that flag is material and reaches core library files, as detailed in Chapter 8. The `tsconfig` also relaxes strictness with `noImplicitAny` set to false. Neither breaks anything today, but both belong on the Phase 2 list so that the build enforces the discipline the codebase already practises in most

places.

4. Architecture and Access Control Verification

4.1 Layered architecture

The separation requested for Phase 1 is real and mechanically enforced rather than conventional. Every API route is built through a single handler factory whose pipeline was verified by source indexing to run in the mandated order: rate limit, authentication, RBAC permission check, zod validation, business logic, uniform envelope, structured logging. A repository-wide search confirmed that no route bypasses the envelope helpers with a bare JSON response. Eight domain module charters under `src/modules` define the future module boundaries, and the cross-cutting engines under `src/lib` give every module a single shared implementation of authentication, storage, auditing, logging, rate limiting, environment access, error semantics and money handling.

4.2 RBAC engine - all eight roles

The permission system is an explicit matrix with wildcard support, evaluated by pure functions. All eight roles resolve their effective permission sets correctly, every permission in the catalog is granted to at least one role, and the checks that matter for the marketplace behave as specified: customers can order and review but see no admin surface; suppliers can manage products but cannot publish services; equipment providers are fenced to equipment; staff roles carry moderation and verification rights. Notably, role assignment is absent from the administrator matrix entirely, making it a super-administrator-only capability by construction.

Probe	Expected	Result
CUSTOMER requests admin:dashboard:view	Denied	403 ForbiddenError
ARTISAN requests identity:users:manage	Denied	403 ForbiddenError
SUPPLIER grants itself marketplace:services:create	Denied by matrix	Not granted
ADMIN requests identity:roles:assign	Denied (super-admin only)	403 ForbiddenError
SUPER_ADMIN requests identity:roles:assign	Allowed	Passes
ADMIN manages a SUPER_ADMIN account	Blocked	403 escalation guard
ADMIN manages an equal ADMIN account	Blocked	403 escalation guard
CUSTOMER or SUPPLIER manages any account	Blocked (non-staff)	403 escalation guard

Table 4.1 - Role boundary spot checks (all passed)

4.3 Privilege escalation - the full 8 x 8 matrix

The canManageRole trust function was exercised for every actor role against every target role, sixty-four pairs in total. The rule set holds without exception: only staff roles manage other accounts at all; nobody below super-administrator may touch a super-administrator account; no role may act on its own level; and permissions are never derived from level arithmetic, only from the explicit matrix. The assignable role sets follow the same shape - the administrator can assign the six non-staff and staff-below roles but never administrator or super-administrator, while the super-administrator can assign all seven others.

4.4 Server-side enforcement and IDOR posture

Authorization is enforced where it belongs. Guard functions consume an AuthContext that only the server session resolver can produce, throw typed errors that the API layer translates into 401 and 403 responses, and log every denial as a security event with actor, role and requested permission. Source inspection confirms that no code path accepts a user identifier or role from request input; resource-level ownership checks are documented as a load-then-verify pattern to be applied when domain resources arrive in Phase 2, and the guard layer is already shaped for it.

5. Security and Data Protection Verification

5.1 Password hashing

Credentials use scrypt through node:crypto with a random sixteen-byte salt per hash and a self-describing storage format that records N, r and p alongside the digest, so parameters can be strengthened later without invalidating existing hashes. Verification re-derives with the stored parameters and compares with a constant-time function; probes with malformed stored hashes, wrong passwords and empty inputs all fail closed. A strength assessor enforces the length and character-class policy at every registration entry point.

5.2 Rate limiting and network edge

The limiter is an in-memory fixed-window implementation with a generic bucket key and preset policies. In development it is disabled by configuration default, which the review confirmed by observing the bypass signature; a subprocess run with the limiter enabled proved enforcement, with the fourth request in a three-per-minute bucket blocked and a sixty-second Retry-After returned while an adjacent bucket remained unaffected. A proxy layer adds coarse per-IP protection for all API paths, security headers on every response, and request-id propagation for log correlation. Both the proxy and the audit helper trust the forwarded-for header, which is safe behind the fronting proxy this deployment assumes but should be pinned to the proxy chain explicitly before any direct exposure.

Header	Value	Origin
X-Content-Type-Options	nosniff	proxy + next.config
X-Frame-Options	DENY	proxy + next.config
Referrer-Policy	strict-origin-when-cross-origin	proxy + next.config
Permissions-Policy	camera=(), microphone=(), geolocation=(self), payment=(self)	proxy + next.config
X-DNS-Prefetch-Control	on	proxy
Strict-Transport-Security	Production only, 2 years, preload	proxy
x-request-id	UUID, echoed to clients and logs	proxy

Table 5.1 - Security headers observed on live responses

5.3 Input validation, uploads and traversal

All external input passes zod schemas before business logic; failures return a structured validation error with per-field details. JSON bodies are capped at one megabyte with the size checked from the

content-length header and the raw payload, and malformed JSON is rejected as a bad request rather than crashing. Upload validation enforces per-category MIME allow-lists and size ceilings, sanitises filenames to a safe stem, and generates keys as category, year, month, UUID and extension, so client data never influences the physical path. At runtime the local storage driver refused every traversal probe, including a parent-directory write attempt and an encoded read attempt, while a legitimate write-read-delete round trip succeeded with the file created owner-only (0600). Magic-byte content sniffing is documented as deferred to the authenticated upload routes of Phase 2, which is the correct layer for it.

5.4 Logging, errors and secret hygiene

The structured logger redacts sensitive keys recursively before serialisation; an intercepted output stream confirmed password, API key and nested token values rendered as redacted while a deliberately safe sibling value passed through. Stack traces are development-only. Unknown errors collapse into an opaque internal error whose public message was probed to confirm it carries no internal text, while Prisma known errors map to semantically correct 409 and 404 responses and other Prisma failures surface as a service-unavailable without driver detail. A pattern sweep across tracked files and full git history found no keys, tokens or credential material; the single hygiene finding is the tracked .env file addressed in Chapter 9. The audit trail is an append-only database table written exclusively through one service that redacts metadata and never throws into the caller, and operational logging is kept separate from it by design.

6. Database, Money and Ghana Data Verification

6.1 Schema review

The Prisma schema replaces the scaffold demo models with two foundation models that demonstrate every convention the Phase 2 schema must follow. Primary keys are collision-free cuid values, the user email carries a unique constraint, timestamps are automatic, soft deletion is available through a nullable deletion timestamp, and lifecycle status is an explicit string field validated at the application boundary. Every column used in lookups or ordering is indexed, including a composite index on the audit entity reference, and table names are mapped to snake_case plurals. The audit log is deliberately append-only: the codebase writes it exclusively through one service and never updates or deletes rows.

Requirement	User model	AuditLog model
Primary key	cuid @id	cuid @id
Indexes	role, status, email	actorId, action, entity pair, createdAt
Timestamps	createdAt, updatedAt, lastLoginAt	createdAt
Soft delete	deletedAt nullable	Not applicable (append-only)
Status field	ACTIVE / SUSPENDED / DEACTIVATED	Action-typed entries
Uniqueness	email unique	Not applicable

Table 6.1 - Schema conventions confirmed

6.2 PostgreSQL migration readiness

The development database may remain SQLite, and nothing in the schema or application code will obstruct the future move to PostgreSQL. The sweep found exactly one raw SQL statement, a portable SELECT 1 health probe; there are no SQLite-specific native type annotations, no Prisma enums (statuses are boundary-validated strings, which is also why the schema is portable), and no database-native JSON columns. Migration is therefore a datasource provider change plus a connection string, followed by the Phase 2 schema work itself.

6.3 Money handling - integer pesewas

Monetary amounts are integers in minor units everywhere, and the finance helpers are the only sanctioned conversion path. Precision probes passed in both directions: decimal strings convert exactly, float inputs round to the correct minor unit, negative and non-numeric inputs are rejected, formatting produces the two-decimal cedi presentation, and the round trip from minor units to a display value and back is lossless. Integer addition and multiplication were exercised at marketplace-relevant scales and are exact well below the safe-integer boundary. One forward-looking note: no division helper exists yet, so when Phase 2 introduces splits, commissions or refunds it must define an explicit remainder policy rather than dividing ad hoc.

Probe	Result
String '19.99' to pesewas	1999 exact
Float 0.1 and '0.07' to pesewas	10 and 7 exact
Rejects negative and NaN input	Throws as designed
formatCedi rendering	GHS two-decimal currency format
pesewas to cedis to pesewas roundtrip	Lossless on 123,456,789
Integer multiply at scale (unit x 45,000 qty)	Exact

Table 6.2 - Money verification probes (all passed)

6.4 Ghana reference data

The market foundation is nationwide rather than capital-centric: all sixteen administrative regions are present as stable constants, the authoritative location hierarchy is documented for Phase 2 database seeding, and no location logic is hard-coded into UI components. Phone validation accepts local and international forms, tolerates spacing, normalises to the international format and rejects letter garbage and overlong inputs. One laxity was found and is registered as a minor issue: the network-digit group in the regular expression is optional, so a nine-digit sequence with an invalid zero-one prefix passes validation. The fix is a one-character change that makes the prefix digit mandatory.

7. API, Storage and Environment Verification

7.1 API foundation - live behaviour

Live probes confirmed the documented conventions hold on real traffic. The health endpoint answers with the standard success envelope, an up-to-one-megabyte body limit, a raised rate-limit ceiling appropriate for monitoring, and a genuine database connectivity check; the API index self-describes the envelope, pagination contract, authentication model and the sixteen planned route groups with only the two foundation endpoints marked operational, which is an honest Phase 1 posture. Pagination parsing clamps out-of-range and malformed input to safe values instead of throwing, and metadata computation was verified for total, page counts and next-previous flags.

Request	Observed
GET /api/health	200; success envelope; database connected; all security headers; x-request-id present
GET /api	200; conventions and 16 route groups documented; only foundation routes operational
Unknown /api path	HTML 404 page rather than the JSON error envelope (see Chapter 9)
Rate-limit enforcement	429 with Retry-After when enabled (subprocess proof); dev default off

Table 7.1 - Live API probes

7.2 File storage abstraction

Storage is interface-first: the application depends on a provider interface with put, get, delete, exists and public-url operations, and a factory selects the driver from configuration. The local driver is the only implementation and fails loudly if a cloud driver is requested before it exists, which prevents silent misconfiguration. Moving to an S3-compatible provider later is therefore a configuration and new-driver change, not a rewrite of business logic. Two small latent defects were noted for the record: the streaming helper casts a Node stream to a web stream without conversion, which will misbehave in Phase 2 download routes until it uses the proper conversion, and streaming is not yet declared on the interface itself.

7.3 Environment configuration and repository hygiene

The environment module validates the entire variable surface with schemas at import time, distinguishes development, staging and production, and enforces that the auth secret must exist outside development while keeping third-party integrations optional so contributors can boot locally. The template file documents every variable with guidance and contains no real values. The ignore policy covers env files, uploads, database files and logs, with the template explicitly un-ignored. The single hygiene defect is historical: a real .env from the initial scaffold commit remains tracked despite the policy; its contents were verified across all history to be a local database path only, and the fix is a single untrack-and-commit.

8. Frontend and Test Suite Results

8.1 Browser verification

The application was driven in a real browser at desktop and mobile viewports. The landing experience renders the Dwellers brand system end to end: brand mark and wordmark, the Find, Buy, Build hero, the eight-module and eight-role sections with provider and staff badges, and the security foundation section. Semantic structure is sound, with a labelled primary navigation, regional landmarks and a correct heading hierarchy. Anchor navigation works, the responsive navigation variant appears on mobile, unknown page routes render a proper not-found state, and the console captured only development-server notices with zero errors or page exceptions. Screenshots are archived with the review artifacts.

Check	Desktop 1440	Mobile 390
Renders without page errors	Pass	Pass
Console errors	None	None
Responsive navigation	Primary nav	Section nav variant
Anchor navigation	Pass	Pass
Broken route handling	Clean 404	Clean 404
Dwellers branding and metadata	Present	Present

Table 8.1 - Frontend checks

8.2 Test suites

Lint is clean across the repository including the review's own audit script after its imports were corrected. The project's foundation suite passes all forty-eight checks, and the independent adversarial suite passes all 109. The one failing gate is the type checker: tsc reports roughly sixteen errors, which the Next build currently ignores by configuration. They fall into three groups: two genuine missing type imports inside core library files, a handful of typing mismatches against newer library signatures that have no runtime effect, and scaffold noise from example and tooling directories that the compiler reaches because they are not excluded. All are addressed by the condition in Chapter 9.

Gate	Command	Result
ESLint	bun run lint	Clean
Foundation suite	bun run verify	48 / 48
Independent gate suite	bun scripts/gate-review.ts	109 / 109
Rate limiter enforcement	Subprocess, limiter on	Blocks on 4th request, Retry-After 60s
Type check	tsc --noEmit	FAIL, approx. 16 errors (masked at build)
Browser E2E	agent-browser session	Zero errors, desktop and mobile

Table 8.2 - Test and quality gate results

9. Issues Register

Ten issues were registered. There are no critical findings. The single high-severity item is the gap between the documented build-error debt and its true scope: because the build ignores type errors, the two missing type imports in core authorization and audit files would never surface at build time, and the safety net the team believes it has is not actually switched on. The recommended sequence is to absorb conditions one to four as the first tasks of Phase 2; none of them blocks the gate, but none should survive past the first Phase 2 increment either.

Sev.	Location	Problem	Recommended fix
HIGH	next.config.ts; src/lib/auth/permissions.ts; src/lib/audit.ts; 5 more files	tsc --noEmit fails with about 16 errors, masked by ignoreBuildErrors: true. Includes missing 'Role' type import in permissions.ts and missing 'LogEventName' import in audit.ts; plus zod v4 refine signature, readonly assignment, generic variance; plus scaffold noise from examples/ and skills/ reached by tsc.	Add the two missing imports; fix the typing mismatches; exclude examples/, skills/, download/ in tsconfig; then set ignoreBuildErrors: false. First task of Phase 2.
MEDIUM	.env (tracked since initial commit)	A real .env is committed, defeating the ignore policy. Contents verified across all history to be a local SQLite path only - no secret exposure.	git rm --cached .env and commit; verify with git ls-files.
MEDIUM	Unknown /api/* paths	Unknown API paths return the default HTML 404 page instead of the JSON error envelope promised by the API index.	Add a catch-all API route returning the standard 404 envelope; natural fit for Phase 2 API work.
MEDIUM	src/lib/db.ts	Prisma query logging is enabled unconditionally, so production would log every statement - a performance cost and a potential PII channel.	Log queries in development only; warn and error everywhere.

Sev.	Location	Problem	Recommended fix
LOW	src/lib/constants/ghana.ts:42	Phone regex makes the network digit optional; probe confirmed '012345678' and '+23312345678' pass validation.	Make the prefix digit mandatory: one-character regex change plus test update.
LOW	src/lib/storage/validation.ts	SVG is allowed for business logos; inline SVG can carry scripts and becomes a stored-XSS vector once serving routes exist.	Drop SVG or sanitize it and serve with content-disposition attachment.
LOW	src/lib/storage/local.ts	stream() casts a Node Readable to a web ReadableStream without conversion; latent defect for Phase 2 download routes, and streaming is not on the provider interface.	Use the proper stream conversion and declare streaming on the interface.
LOW	src/lib/rate-limit.ts	At the bucket cap, eviction only removes expired buckets; a flood of unique keys can outpace it and grow memory on a single node.	Hard-evict oldest entries after the sweep when at capacity.
LOW	rate-limit.ts; audit.ts (IP extraction)	Forwarded-for header trusted unconditionally; spoofable when not behind the trusted proxy, weakening buckets and audit IP values.	Trust only the known proxy chain; document the deployment requirement.
LOW	next.config.ts; tsconfig; password params	Inherited smells: reactStrictMode off, noImplicitAny off, ignore entries named 'test' and 'prompt', script N below current OWASP guidance.	Enable both flags, rename ignore entries, plan an N bump (self-describing hash format supports it).

Table 9.1 - Issues register

10. Production Risks

Beyond discrete issues, six structural risks deserve explicit tracking because they become real the moment the platform takes production traffic or real users. None is a Phase 1 defect; each is a documented deferral whose timing now has a named owner phase. The register below pairs every risk with its mitigation path so that Phase 2 planning can sequence them deliberately rather than rediscover them under load.

Risk	Why it matters	Mitigation path
Type safety not enforced at build	Regressions can ship untyped; the flag hides new errors as well as old ones.	The high-severity condition at the start of Phase 2; then keep the flag off permanently.
No Content-Security-Policy yet	XSS defence currently rests on framework escaping alone.	Introduce CSP with nonces on authenticated surfaces in Phase 2, as already documented.
JWT role staleness up to 7 days	A revoked role remains honoured until token refresh; the re-check pattern is documented but not yet implementable.	Implement live DB re-check on sensitive actions with the Phase 2 user store.
In-memory rate limiting	Single-node assumption; horizontal scaling silently drops protection.	Swap the Redis-backed store behind the existing interface before multi-instance deploy.
Declared upload types only	Content-type is taken from the request until magic-byte sniffing lands.	Sniff bytes in the authenticated upload routes of Phase 2.
SQLite only in development	Fine for now; production must not ship on it.	Migrate provider at Phase 2 schema delivery; no blockers found (Chapter 6).

Table 10.1 - Production risk register

11. Phase 2 Readiness

YES Ready for Phase 2	1 HIGH Condition, type-check enforcement	4 MEDIUM Conditions, hygiene and API	5 LOW Conditions, small and safe
---------------------------------	--	--	--

Dwellers is ready to proceed to Phase 2 - Database Schema and Backend Foundation. The gate found no architectural defect, no missing foundation, and no security hole in what Phase 1 was scoped to deliver; the reported implementation exists, behaves, and in several places exceeds the brief, notably the adversarially-verified escalation defence and the honest, convention-first API surface. Readiness is conditional in exactly one sense: the ten registered issues should be folded into the Phase 2 plan as first-class work items rather than carried as background debt.

The recommended Phase 2 opening sequence, in order: enforce type checking (the high-severity condition) so that everything built afterwards is typed at build time; untrack the environment file and make query logging environment-aware; add the API catch-all envelope while building the first real endpoints; and apply the one-line hardenings - phone regex, strict-mode flags and proxy-pinned client IPs - alongside them. With those absorbed, Phase 2 can proceed on a foundation whose guarantees are enforced by code, not by claims.

Per the gate's terms of reference, Phase 2 has not been started. The next instruction from the project owner authorises the transition.